# Does longer context help a small character-level transformer?

Daniel Moody - 23370157
Nandakishore Vinayakrishnan - 23070854

Team - Eoin-Please-Give-Us-An-A1




LM173 BSc/MSc Immersive Software Engineering

Faculty of Science & Engineering

October 2026

**Research question** **2**
**Motivation** **2**
**Reference condition** **2**
Character N-gram 2
Transformer with Context Length set to 32 2
**Controlled comparison** **2**
**Dataset and split strategy** **3**
**Evaluation method** **3**
**Repeated-run results** **4**
**Error or failure analysis** **6**
Where longer context helps and where it hurts 6
Loss by position in the window 7
Training dynamics 8
Failure case: fold 9 on the test set 9
**Interpretation** **10**
Answer to the research question 10
What the evidence does and does not support 10
Why there is no overall effect 11
What remains uncertain 11
Relation to the motivation 11
**Limitations and threats to validity** **12**
**Compute** **12**
**Provenance and licensing** **12**
**Ethical Considerations** **12**
**Use and verification of external tools** **13**

# Research question

With training characters held fixed, does increasing context length from 32-128-256 lower test bits-per-character for a 4-layer character transformer on TinyShakespeare by more than the variation across matched runs?

# Motivation

Context length is how many previous characters the model can see when it predicts the next one. A longer context window costs more compute: the attention cost per sequence grows with the square of the context length (linearly per character). At our model size this is small in practice, since a context-256 run took about 2.12 minutes against 1.92 minutes for context 32 on a T4. Longer context is still only worthwhile if the model actually uses text that far back.

# Reference condition

We used two reference conditions in this project.

## Character N-gram

This model predicts the next character from counts of what followed the previous *n-1* characters during training. It can show what plain, local statistics can achieve without any deep learning, to give us a baseline the transformer should surpass, and to show that it learns more than local statistics. Without it, a null result could simply mean the transformer was broken; beating the n-gram rules that out.

## Transformer with Context Length set to 32

This is the model with the shortest context length of our given set. Its architecture is identical to the 128 and 256 models apart from the size of the position-embedding table (1.80M parameters at context 32 against 1.84M at context 256). On each fold, all three models start from identical weights apart from that table. This reference condition helps us understand that any difference from this transformer can reasonably be attributed to context length.

# Controlled comparison

The experiment called for the only variable that changes to be the context length, between 32,128,256 characters. To keep the amount of training text equal, every step will see a total of 16,384 characters. To compensate, the batch size shrinks with the context length: 16,384 / L, i.e. 512, 128 and 64 windows for contexts 32, 128 and 256.
The model architecture is based on the "baby GPT" configuration Karpathy uses for TinyShakespeare in nanoGPT (6 layers, 6 heads, width 384), deliberately scaled down to fit our compute and development time.

It incorporates a GPT-style decoder with 4 heads, 4 layers, width 192, dropout 0.1. The optimiser chosen was AdamW, with a learning rate set to a cosine schedule from 1e-3 to 1e-4, similar to nanoGPT's architectural decisions.

On a given fold of the data, all 3 context lengths use the same seed: fold f uses seed f, giving 10 runs per context length (30 transformer runs in total). This ensures that every comparison is paired, ensuring that each run has the same fold, same seed, and the same starting point.

# Dataset and split strategy

TinyShakespeare is 1,115,394 characters of Shakespeare's plays, with a vocabulary of 65 characters.

Of the 1.1M characters, the last 10% (~110k characters) is saved as a test set. The first 90%(~1M characters) is cut into 10 contiguous folds of about ~100k characters. One block of the 10 is used for validation, while the rest 9 are used for training. Each run therefore trains on about 81% of the text and validates on about 9%, with the same 10% test set for every run. Across the 10 folds, every part of the first 90% is used for validation exactly once. The text was split to ensure that splits only occur during line breaks, so no line gets split across sets.

# Evaluation method

The primary metric used was Bits-Per-Character (BPC). BPC is the average number of bits the model needs to encode each character. It equals cross-entropy in nats divided by ln 2. It is our evaluation metric; the models themselves are trained to minimise cross-entropy averaged over every position in the window.

Every model will predict the same characters from the 257th character of a fold onward. This way, we can ensure every option for context length, up to the 256-context length option, has a full window of information to work off of.

Let "L" be the context length for each run, then a model with context L would see the L characters before each target, and score only the final position of each window. So each prediction has exactly L characters of history, and all models are judged on the same characters.

This method of prediction matters because training loss is an average over every position in a window, and early positions have almost no context. That average makes the context-32 model look worse simply because it tends to have more low-context positions, not because it actually predicts worse.

Every 250 steps, each transformer's validation loss is estimated on 20 fixed validation batches, keeping the checkpoint with the lowest value.

Since fold difficulty can vary to an extent that could affect our measurements, each comparison is made within a given fold. For each fold, we subtract the shorter-context model's BPC from the longer-context

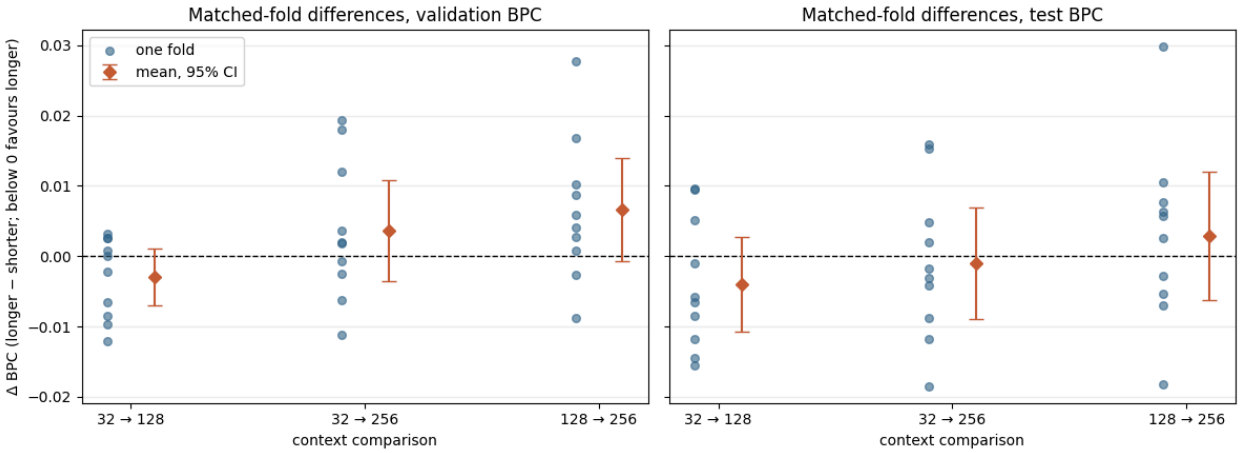
model's BPC on that fold, and call it Δ. If Δ is negative, the longer-context option beats the shorter-context option for that fold. Both models face the same text with the same seed, cancelling out the difficulty. We then report the mean Δ, a 95% t-interval, and how many folds favour the larger context.

# Repeated-run results

No context length was consistently better on either validation or test. The transformer beat the n-gram by about 0.36 BPC on test and 0.30 on validation, on every fold.

| Condition | Test BPC, mean (SD) | Test range | Validation BPC, mean (SD) |
|---|---|---|---|
| Context 32 | 2.1335 (0.1087) | 2.0866–2.4420 | 1.9191 (0.0801) |
| Context 128 | 2.1295 (0.1061) | 2.0819–2.4301 | 1.9161 (0.0792) |
| Context 256 | 2.1324 (0.1077) | 2.0809–2.4378 | 1.9227 (0.0761) |
| N-gram (tuned per fold) | 2.4954 (0.1315) | 2.4454–2.8689 | 2.2215 (0.0904) |

The upper end of each test range (about 2.44 for the transformers and 2.87 for the n-gram) comes entirely from fold 9; this is explained in the error analysis and does not affect the paired comparisons.




The spread across folds (SD about 0.11 on test set) is around 30 times larger than any mean difference. Pairing runs by fold removes most of that spread, but the remaining spread of the differences (SD about 0.01) is still larger than the effects themselves. Paired t-tests on test gave p = 0.22 (32 -> 128), 0.78 (32 -> 256) and 0.49 (128 -> 256).

| Comparison | Mean Δ | 95% CI | Validation folds favouring longer contexts |
|---|---|---|---|
| 32 → 128 | −0.0030 | −0.0070 to +0.0011 | 5/10 |

| | | | |
|---|---|---|---|
| 32 → 256 | +0.0036 | −0.0036 to +0.0108 | 4/10 |
| 128 → 256 | +0.0066 | −0.0008 to +0.0139 | 2/10 |

The difference in sign between context_32 and context_256 differs between validation and test, which could be a sign that we are measuring noise. The closest thing to a measurable effect is that context_256 is worse than context_128 in 8 of 10 folds. The test set does not confirm it, as only 6 out of 10 folds favoured the shorter context_128, which could be attributed to a random choice influenced by noise.

Below is the BPC of every run, with the n-gram's chosen order and discount (d) for each fold. Every transformer run chose its checkpoint at step 3,000, except context 32 on fold 0 and contexts 128 and 256 on fold 3, which chose step 2,750.

| Fold | Val 32 | Val 128 | Val 256 | Test 32 | Test 128 | Test 256 | N-gram (order, d) | N-gram val | N-gram test |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1.9458 | 1.9483 | 1.9395 | 2.0927 | 2.0863 | 2.0809 | 5, 0.75 | 2.2356 | 2.4483 |
| 1 | 1.8337 | 1.8253 | 1.8355 | 2.0866 | 2.0962 | 2.1025 | 5, 0.75 | 2.1146 | 2.4454 |
| 2 | 1.7897 | 1.7923 | 1.8091 | 2.0911 | 2.0962 | 2.0893 | 6, 0.9 | 2.0790 | 2.4752 |
| 3 | 1.9327 | 1.9230 | 1.9507 | 2.0964 | 2.0819 | 2.1116 | 5, 0.75 | 2.2684 | 2.4522 |
| 4 | 1.9501 | 1.9502 | 1.9476 | 2.1063 | 2.1005 | 2.1031 | 5, 0.75 | 2.2521 | 2.4526 |

| Fold | Val 32 | Val 128 | Val 256 | Test 32 | Test 128 | Test 256 | N-gram (order, d) | N-gram val | N-gram test |
|---|---|---|---|---|---|---|---|---|---|
| 5 | 1.9154 | 1.9162 | 1.9190 | 2.1150 | 2.0994 | 2.0966 | 5, 0.75 | 2.2233 | 2.4521 |
| 6 | 1.8466 | 1.8498 | 1.8586 | 2.0955 | 2.0871 | 2.0976 | 5, 0.75 | 2.1627 | 2.4480 |
| 7 | 2.0322 | 2.0257 | 2.0315 | 2.1069 | 2.1164 | 2.0982 | 5, 0.75 | 2.3266 | 2.4610 |
| 8 | 1.9086 | 1.8966 | 1.8974 | 2.1020 | 2.1011 | 2.1068 | 5, 0.75 | 2.1813 | 2.4508 |
| 9 | 2.0361 | 2.0339 | 2.0380 | 2.4420 | 2.4301 | 2.4378 | 5, 0.9 | 2.3710 | 2.8689 |

# Error or failure analysis

The overall bits per character (BPC) hides what longer context changes. To see where it helps or hurts, we split the validation scores in three ways: by the kind of character being predicted, by position within the window, and by fold.

## Where longer context helps and where it hurts

Each scored character was assigned to one of five categories. For each fold, we then compared the 128 and 256 models with the context-32 model on the same characters. Negative Δ means the longer context is better; "folds better" counts the folds where the longer context had the lower BPC.

| Character type | Share of targets | Context 32 BPC | Δ 128 − 32 (folds better) | Δ 256 − 32 (folds better) |
|---|---|---|---|---|
| Line start (first letter of a line) | 2.2% | 4.284 | −0.062 (9/10) | −0.160 (10/10) |
| Non-letter (space, punctuation, newline) | 21.6% | 1.291 | −0.054 (10/10) | −0.053 (10/10) |
| Speaker heading (e.g. BAPTISTA:) | 9.6% | 1.461 | −0.011 (5/10) | −0.049 (9/10) |
| Word start | 15.1% | 3.968 | −0.003 (8/10) | +0.005 (2/10) |
| Mid-word letter | 51.5% | 1.564 | +0.023 (0/10) | +0.043 (0/10) |

Observed:

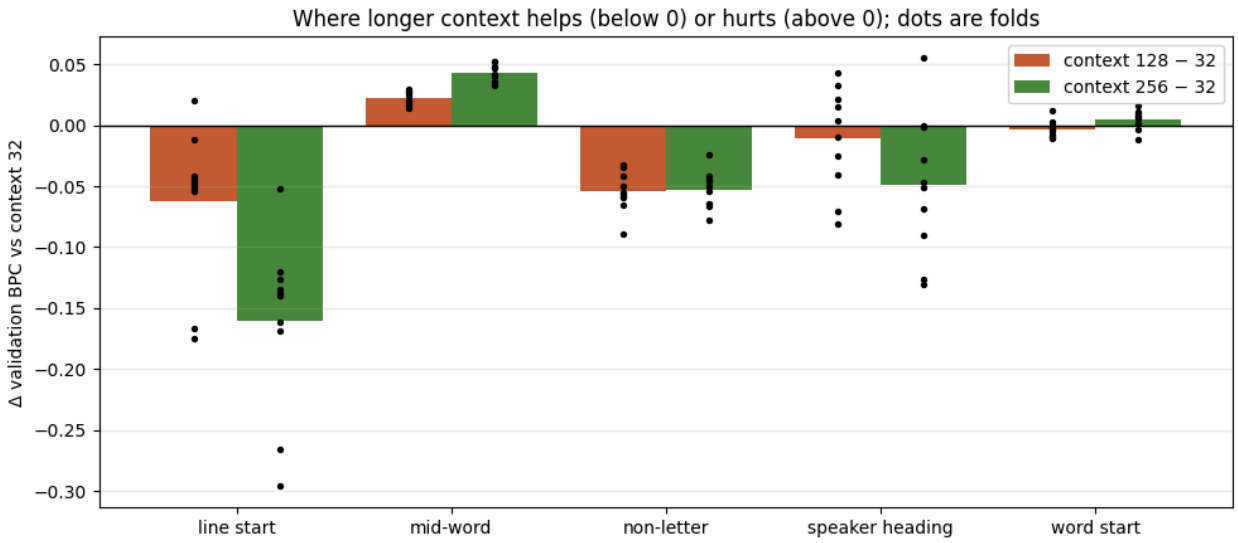
- Gains: longer context consistently lowered BPC on the structural parts of the text: the start of a new line, spaces and punctuation, and with 256 characters, speaker headings.
- Losses: on letters in the middle of words, both longer contexts were worse than context 32 in every one of the 10 folds.
- They cancel out: mid-word letters are half of all targets. Weighting each category by its share gives an overall change of about −0.003 for 128 and +0.003 for 256, which matches the overall results.

The null result is therefore not "context does nothing". It is two real, consistent effects in opposite directions that cancel.

Inferred:

- Why longer context helps structure: the start of a line and the next speaker depend on text further back than 32 characters. Examples are the rhythm of a verse line, or who has been speaking. Only the longer models can see that far.
- Why it hurts spelling: under a fixed budget of 16,384 characters per step, the longer models see far fewer independent windows per step (64 at context 256, against 512 at context 32). They appear to have learnt local spelling patterns less well in the same number of steps.

Uncertain: we did not test this explanation directly. A run with a longer training budget would show whether the mid-word loss shrinks.




## Loss by position in the window

We also measured validation BPC at every position in the window, which is the loss when the model has seen 1, 2, …, L characters of context. The values are means over the 10 folds.

| Characters of context available | 1 | 8 | 32 |
|---|---|---|---|
| Context 32 model | 3.566 | 1.964 | 1.888 |
| Context 128 model | 3.594 | 1.982 | 1.895 |
| Context 256 model | 3.610 | 2.008 | 1.917 |

Observed:

- Worse at every position up to 32: given the same amount of context, the longer-context models are slightly worse than the context-32 model at every position up to 32.
- Small improvement after that: beyond 32 characters, their loss keeps falling, but only slightly. Over its last 32 positions, the 256 model averages 1.917 BPC, roughly the same as the context-32 model at its last positions.

Inferred: the extra context is used, but mostly to make up for a weaker short-range model rather than to improve on it.

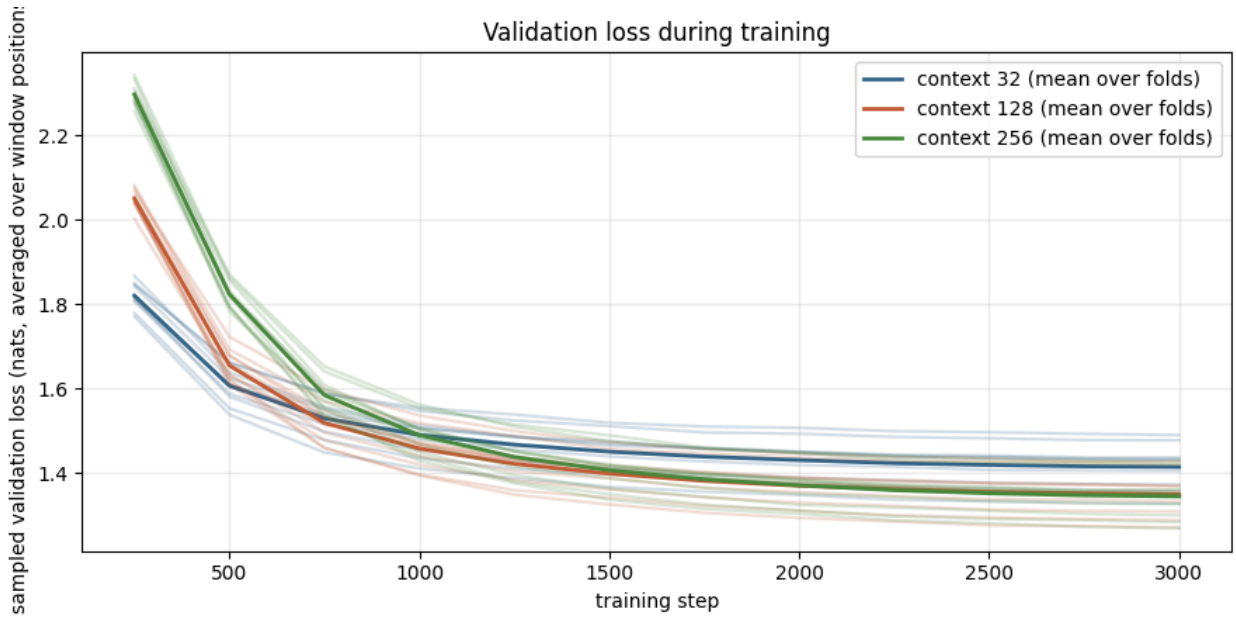
This also explains why the training curves are misleading. There, context 256 appears clearly better (a final validation loss of 1.345 nats against 1.415 for context 32). That loss is averaged over all window positions, and a long window contains proportionally fewer of the poorly-informed early positions. This is why we report fixed-target BPC rather than training loss.

## Training dynamics

Observed:

- Slow start for longer contexts: at step 1,000, the 256 model's mean validation loss (1.490) was no better than context 32's (1.491), despite the averaging advantage described above.
- Still improving at the end: 27 of the 30 runs chose their checkpoint at the final step (3,000), so the models were still improving when training stopped.

Inferred: longer-context models learn more slowly early on, and our results describe this fixed budget rather than fully trained models.




## Failure case: fold 9 on the test set

Observed: on the test set, every model trained for fold 9 scored far worse than the same model on any other fold:

- Transformers: about 2.43–2.44 BPC against 2.08–2.12.
- N-gram: 2.87 against about 2.45.

On validation, fold 9 (about 2.04 BPC) is in line with fold 7 (about 2.03).

Explanation: the test set begins partway through The Taming of the Shrew; its first line is "GREMIO: Good morrow, neighbour Baptista." The opening of that play sits in fold 9's block, which contains 105 speaker headings belonging to the test set's main characters (Petruchio, Katharina, Baptista and others). No other block contains any.

- Folds 0–8: these models trained on fold 9's block, so they had already seen the start of the play they were tested on.

- Fold 9: this model held that block out for validation, so it met the play for the first time at test time.

There is no overlap in the text itself, and the test set was never used to make a decision. However, the test scores for folds 0–8 reflect familiarity with the play, so fold 9's score is the more honest estimate for unseen text.

Effect on our conclusion: the main comparison is made within each fold, so this offset cancels in every Δ, and fold 9's own differences (−0.012, −0.004, +0.008) look like the other folds'. It does mean the absolute test BPC, and its standard deviation of about 0.11, are inflated by one unusual fold. We discuss this further under limitations.

# Interpretation

## Answer to the research question

Under the conditions we tested (a 4-layer, 1.8M-parameter character transformer trained for 3,000 steps of 16,384 characters each), we found no convincing evidence that increasing the context length from 32 to 128 or 256 characters lowers test bits-per-character by more than the variation across matched runs.

Observed: on the test set, the mean paired differences (longer minus shorter context) were:

| Comparison | Mean Δ test BPC | 95% CI | Folds favouring longer context |
|---|---|---|---|
| 32 → 128 | −0.0039 | −0.0107 to +0.0028 | 7/10 |
| 32 → 256 | −0.0010 | −0.0089 to +0.0069 | 6/10 |
| 128 → 256 | +0.0029 | −0.0062 to +0.0121 | 4/10 |

- All three intervals include zero, on test and on validation.
- No comparison favoured the longer context consistently across folds.
- The largest mean difference (about 0.004 BPC) is under 0.2% of the BPC itself, and smaller than the spread of the paired differences (SD about 0.01).

The comparison itself was informative. Both reference conditions behaved as expected: every transformer beat the tuned n-gram on every fold, by about 0.30 BPC on validation and 0.36 on test. So the transformers learnt far more than local character statistics, and the null result is not caused by a broken or untrained model.

## What the evidence does and does not support

Supported:

- At this model size and training budget, moving from 32 to 128 or 256 characters of context made no meaningful difference to overall BPC.
- The intervals also bound how large any benefit could plausibly be. On test, they rule out (as a rough guide, see limitations) improvements larger than about 0.011 BPC for 128 and about 0.009 BPC for 256 over context 32.

Not supported:

- "Context 128 is better than context 32." There is a small advantage for 128 on both validation (−0.0030) and test (−0.0039), but it was not consistent across folds (5/10 and 7/10) and both intervals include zero. A difference of this size could easily be noise.
- "Longer context does not matter for this task." The error analysis shows that it matters in both directions; the two effects cancel in the overall score.
- Any claim about larger models, longer training or other datasets. We did not test these.

## Why there is no overall effect

Inferred: The error analysis suggests the null result comes from a trade-off rather than from the model ignoring distant text:

- Longer context consistently helped with the structure of the play, since the first letter of a new line (−0.160 BPC at context 256), spaces and punctuation (about −0.05), and speaker headings (−0.049 at context 256). These depend on information more than 32 characters back, such as the length of a verse line or who has been speaking.
- Longer context consistently hurt the prediction of letters inside words (+0.023 at 128 and +0.043 at 256, worse on every fold). These letters are half of all targets, so the losses cancel the gains.

Our best explanation is the fixed training budget. To keep the characters per step equal, the context-256 model sees only 64 independent windows per step, against 512 for context 32. Its training curves show it learnt more slowly at the start, and the loss-by-position results show it was slightly worse than the context-32 model even when given the same amount of context. In other words, the longer models gained the ability to use distant text but had a weaker short-range model, and after 3,000 steps these roughly balanced out.

## What remains uncertain

- Whether the trade-off is caused by the training budget. 27 of 30 runs were still improving at the last step. With longer training, the mid-word penalty might shrink and the structural gains might remain, which would favour longer context; or the context-32 model might keep improving as well. We cannot predict the direction confidently from our data.

- Whether the small advantage for 128 over 32 is real. It points the same way on validation and test, but it is well within the run-to-run variation. Ten folds are not enough to detect an effect of this size.
- How well the absolute test BPC reflects truly unseen text. As explained in the error analysis, 9 of the 10 models had seen the opening of the play the test set continues, so absolute test scores are probably optimistic. The paired comparisons are not affected, because both models in each pair saw the same training text.

### Relation to the motivation

Longer context only pays off if the model actually uses text that far back. Our results show that this small model does use it for structural prediction, but that under an equal budget of training characters this does not translate into a better overall score. For a model of this size trained this briefly, the extra context did not justify itself in BPC terms. At this scale it also cost little extra compute (about 10% more training time), so the practical cost of trying it is small.

## Limitations and threats to validity

The main threat to our conclusion is model capacity, and training budget. A larger or longer-trained model might use a longer context where ours did not. The experiment had to be cut short at 3000 steps, and thus didn't hit convergence but was cut short because the validation/training losses barely dropped.

The experiment that would most reduce the remaining uncertainty is to repeat the same matched comparison at a second, larger model size with a larger step size budget.

Additionally, the seeds were arbitrarily chosen. A future experiment would need a more defensible strategy for picking seeds, preferably with more seeds.

## Compute

The whole experiment took ~2 hours of GPU time on a Tesla T4 GPU using Google Colab. It runs on Python 3.13.5. Runs were saved as resumable checkpoints to Google Drive every 250 steps, so a Colab disconnect does not lose finished work. Re-running the notebook with "Run All" would reproduce the results.

## Provenance and licensing

TinyShakespeare is a dataset of 40k lines from Karpathy, which can be found on GitHub at https://github.com/karpathy/char-rnn/blob/master/data/tinyshakespeare/input.txt. The repository is MIT-licensed, and Shakespeare's plays are in the public domain.

The model architecture is inspired by and draws architectural approaches from Karpathy's nano-GPT implementation for the dataset, Shakespeare-char (GPT-style decoder, tied weights, AdamW), found on https://github.com/karpathy/nanogpt.

The model was built using PyTorch, numpy, pandas, SciPy, and matplotlib libraries.

# Ethical Considerations

The data contains no personal, confidential or sensitive information. The models are tiny and produce only Shakespeare-style text, so they have no realistic use for deception or harm. One limitation is that the corpus reflects the language, attitudes and stereotypes of 16th- and 17th-century England, so generated text can reproduce them. That is irrelevant to our metric, but it would matter for any real use of the model.

# Use and verification of external tools

We used Claude as an AI assistant for ideation, explanations, and guidance on model code and final document phrasing . We also built on Karpathy's Shakespeare-char design. Every number in this report comes from results/runs.csv and results/summary.json.

# Contribution

This notebook and document was worked on collaboratively, with both parties working in-person on each other's machines. We both agree that the work was fairly split and that no one was burdened with an unfair amount of work.